

Scribl Artifact Generation —POC Handover

Prepared by: Mission Cloud Services, Inc.

Prepared for: Scribl, Inc.

Engagement: Artifact Generation Proof of Concept (Professional Services)

Date: June 2026

Table of Contents

1. Executive Summary	1
2. What Was Delivered	2
3. End-to-End Workflow	3
4. How It Maps to the Vision	4
5. Time & Cost Factors	5
6. Pricing & Cost Reporting.....	6
7. Tuning & Customization Guide.....	7
8. Areas for Improvement & Future Enhancements.....	8
Quality & fidelity	8
Robustness & operations	9
9. Recommended Next Phase	9
10. Appendix — Running the Pipeline.....	10
<i>One session, end-to-end.....</i>	<i>10</i>
<i>With drop shadows and 3 refinement passes</i>	<i>10</i>
<i>All sessions, consolidated batch report.....</i>	<i>10</i>

1. Executive Summary

Scribl set out to test a single question: **can generative AI assemble the drawings from a game session into a single, shareable keepsake — without redrawing or "interpreting" anyone's art?**

Earlier experiments had taken too much creative liberty, turning a participant's stick-figure dog into something that was no longer theirs.

Over a three-week, time-boxed engagement, Mission built a working pipeline that proves the concept. It takes the raw drawings and prompt from a Scribl session and produces a composed scene image where **every participant's original artwork is preserved pixel-for-pixel** — never regenerated. Generative AI is used only for the connective tissue around the art: understanding what each drawing depicts, generating a hand-drawn-style background that matches the session, and intelligently arranging the cards into a coherent scene.

The result: the original art stays true (the North Star from discovery), the output looks composed rather than like "alphabet soup," and the whole thing runs automatically — removing the manual drag-and-resize work that hosts do today.

This is a proof of concept, delivered as a documented, runnable Python pipeline on AWS Bedrock. It validates feasibility and gives Scribl a concrete foundation — and a clear set of knobs — to decide whether and how to productize.

2. What Was Delivered

Deliverable	Description
6-stage generation pipeline	A modular Python pipeline (`masking/`) that takes a session folder and produces a finished artifact. Each stage runs standalone or end-to-end.
Batch runner	Processes all sessions in one command, with per-stage skip flags for fast iteration.
Per-run reporting	Every run emits a `pipeline_report.md` with timing and token cost per stage — built for Scribl's budget sensitivity.
Externalized configuration	All prompts and tuning parameters live in a single `config.json` — no code changes needed to experiment. Supports per-step model selection and model-specific pricing.
Documentation	`README.md` with full setup, stage reference, and CLI options; this handover document.

Models used (AWS Bedrock):

- **Claude (Opus 4.8 / Sonnet 4.6)** — drawing descriptions, background prompt authoring, composition planning, and iterative review. Each AI stage's model is now configurable independently (see Tuning).
- **Stability AI (Stable Image Inpaint)** — renders the sketch-style background.

Everything runs in the customer's preferred region (`us-east-1`) against Bedrock — no data leaves AWS.

3. End-to-End Workflow

The pipeline runs six stages. Two are pure image processing (no AI); four use foundation models on Bedrock. The model used for each AI stage is configurable in `config.json`.

#	Stage	What it does	Engine
1	Mask	Strips the white card background and gray border, leaving each drawing on a transparent layer. Records quality metrics per card (coverage, blank/dense flags).	OpenCV (no model)
2	Describe	Sends each masked drawing + the session prompt to Claude, which returns a one-sentence description of the subject. Description calls now run in parallel , significantly cutting processing time — a session that used to take ~7 minutes now completes in under 2 minutes.	Claude (Sonnet 4.6)
3	Generate Background	Claude writes a constrained, sketch-style prompt; Stability AI renders a clean background. Hard rules keep it minimal, people-free, and hand-drawn — never photorealistic. Backgrounds now include relevant indoor scenes as well as outdoor landscapes (previously only outdoor).	Claude + Stability AI
4	Plan Composition	Claude views the background and arranges every card — position, scale, z-order, thematic groupings — so the scene reads logically.	Claude

5	Compose	Layers the original drawings onto the background per the plan. Large sessions auto-split across multiple smaller artifacts to avoid clutter (e.g. a 40-person session splits into 4 artifacts of 10 drawings each). The max drawings per artifact is configurable in `config.json`.	Pillow (no model)
6	Refine	Claude reviews the composed image against the background and nudges placement; loops until satisfied or the iteration cap is reached.	Claude

Key design choice: the participants' drawings are only ever *masked and placed* — never sent to an image model for regeneration. Fidelity to the original art is structurally guaranteed, not just prompted for.

Per-step model selection: Each AI stage can be pointed at a different model via `config.json`. For example, the Describe stage (Stage 2) now uses **Claude Sonnet 4.6** instead of Opus 4.8 — improving performance and reducing cost while preserving quality — while higher-judgment stages can remain on Opus.

4. How It Maps to the Vision

The discovery sessions defined what a "good" artifact looks like. Here's how the POC delivers against each point:

Scribl's requirement (from discovery)	How the POC addresses it
Original art preserved — the North Star	Drawings are masked and composited directly. No regeneration, no "creative liberty."
Looks integrated, not "alphabet soup"	Claude plans groupings and placement, then refines iteratively for visual balance.
No AI-realism — keep the hand-drawn feel	Background prompt is locked to "pencil sketch, child-like drawing" style with a strong negative prompt against photorealism.
Background should enhance, not distract	Backgrounds are minimal, muted, and people-free; the drawings remain the most detailed thing on the canvas. Both indoor and outdoor scenes are now supported.

Remove manual host effort	The whole arrangement is automatic — no drag, resize, or reposition by hand.
Shareable output	Standard PNG at a social-friendly canvas size (configurable aspect ratio).

5. Time & Cost Factors

Every run writes a `pipeline\_report.md` breaking down duration and Bedrock token usage per stage, priced from the rates in `config.json`. The figures below are planning-level estimates.

What drives cost and time (per session):

Factor	Impact
Number of drawings	Largest driver. Stage 2 (Describe) makes one Claude call per drawing; more cards = more calls and a longer composition plan. Parallel description calls now keep wall-clock time low even for large sessions.
Refinement iterations (`--refine-iterations`, default 3)	Each iteration is a multi-image Claude call. Stops early when Claude is satisfied, so cost is often below the cap.
Model choice	Opus gives the best planning quality; a smaller/faster Claude model (e.g. Sonnet 4.6) materially cuts cost where quality allows. Models are now selectable per stage (see Tuning).
Background generation	One Claude prompt call + one Stability render per session — roughly fixed per artifact.
Masking & composition	Effectively free — local CPU image processing, no model calls.

Where the model spend concentrates: the AI cost is dominated by the **image-heavy stages** — Describe (one image per card) and Refine (two images per iteration). Output tokens are small; input/image tokens dominate. The two non-AI stages (Mask, Compose) add only local compute time.

Cost-control levers already built in:

- `--refine-iterations` caps the most expensive stage; early-stop avoids wasted calls.
- `--skip-\*` flags let you re-run only changed stages instead of paying for the whole pipeline.
- Stage 2 describe-calls run concurrently, shortening wall-clock time (~7 min → under 2 min on large sessions).
- Per-step model selection lets you route cheaper stages to lower-cost models (e.g. Sonnet 4.6 for Describe).
- Per-stage reporting makes it trivial to spot the costliest step before scaling up.

> **Note on engagement cost:** The Professional Services engagement was a fixed 120-hour, 3-week POC fully offset by AWS funding (net \$0 to Scribl per the SOW). The cost factors above refer to the per-artifact AWS Bedrock runtime cost of operating the pipeline, which Scribl controls via the levers above.

6. Pricing & Cost Reporting

The pipeline emits accurate, per-stage cost reporting on every run. `config.json` supports **model-specific pricing**, so each model's input and output token rates are tracked independently — meaning reports remain correct even when different stages use different models. As Bedrock rates change, you update them in one place (`config.json` → `pricing`) and all reports stay accurate.

Example pricing configured in `config.json`:

Model	Input rate	Output rate
Claude Opus 4.8	\$15.00 / 1M tokens	\$25.00 / 1M tokens
Claude Sonnet 4.6	\$3.00 / 1M tokens	\$15.00 / 1M tokens

Because pricing is tracked per model, a report can blend Opus (planning/refinement) and Sonnet (description) costs and still produce an accurate total.

Representative single-session costs (from sample runs):

Session	Runtime	Input tokens	Output tokens	Total cost
`20 - I am my best self when`	1m 06s	10,608	2,637	\$0.1203
`6 - The bravest thing I have ever done is`	1m 38s	20,860	5,384	\$0.2175

Example per-stage breakdown (session `6 - The bravest thing I have ever done is`):

Stage	Model	Duration	Total tokens	Cost
Stage 1 — Mask	—	0m 00s	—	—
Stage 2 — Describe	Sonnet 4.6	0m 03s	5,742	\$0.0231
Stage 3 — Generate background	Opus 4.8	0m 25s	1,509	\$0.0249
Stage 4 — Plan composition	Opus 4.8	0m 21s	5,118	\$0.1011
Stage 5 — Compose canvas	—	0m 00s	—	—

Stage 6 — Iterative refinement	Sonnet 4.6	0m 47s	13,875	\$0.0684
Total		1m 38s	26,244	\$0.2175

Representative batch run (20 sessions):

Metric	Value
Sessions processed	20
Total runtime	34m 57s
Total input tokens	427,246
Total output tokens	100,701
Estimated total cost	\$4.3106

This works out to roughly **\$0.12–\$0.25 per typical session**, with larger/busier sessions (more drawings, more refinement) landing higher. Prices shown are from ``config.json``; actual Bedrock costs may differ.

7. Tuning & Customization Guide

A core goal of the POC was to make experimentation cheap. **All behavior-shaping levers live in ``masking/config.json`` and CLI flags — no code changes required.**

Lever	Where	What to tune
Prompts	<code>`config.json` → prompts`</code>	Describe, background-style, placement rules, and refinement instructions are all editable text. This is the single highest-leverage place to shape output.
Background style	<code>`generate_background_instruction` / `_negative`</code>	Adjust the art style, allowed scene types (indoor/outdoor), or strictness. Swap in Scribl's own style language or brand guidelines here.
Per-step model	<code>`config.json` → model_id` (per stage) or <code>`--model`</code></code>	Select which model runs each AI stage independently — trade quality vs. cost/latency by routing each stage to the right Claude tier (e.g. Sonnet 4.6 for Describe, Opus 4.8 for planning).
Canvas / aspect ratio	<code>`--canvas-width` / <code>`--canvas-height`</code></code>	Target specific social formats (square, portrait, landscape).
Drawings per artifact	<code>`max_drawings_per_image`</code>	Controls when a busy session splits into multiple artifacts

		(e.g. 40 drawings → 4 artifacts of 10).
Refinement depth	<code>--refine-iterations`</code>	More iterations = tighter layout, higher cost.
Drop shadows	<code>--shadow`</code>	Adds depth under each card.
Pricing	<code>config.json → pricing`</code>	Model-specific input/output rates. Keep aligned with current Bedrock rates so reports stay accurate.
Masking thresholds	<code>mask_pipeline.py` (HSV value/saturation)</code>	Rarely needed; tune only if specific card styles mask poorly.

Most impactful areas to tune next: background style prompts (to nail Scribl's aesthetic), placement rules (scene logic), and per-stage model selection (cost vs. quality).

8. Areas for Improvement & Future Enhancements

The POC proves feasibility; the following would raise quality and unlock the broader vision discussed in discovery.

Quality & fidelity

- **Multi-scene backgrounds.** Today the AI generates a single background scene per artifact (e.g. a kitchen *or* a park) — you get one scene or the other. A future enhancement would test prompts that generate **multiple scenes in one image** (e.g. a kitchen *and* a park together), so drawings that belong indoors are placed inside and outdoor drawings are placed outside. This needs further prompt testing.
- **AI grouping of drawings.** Add a dedicated AI round that **groups related drawings together before placement**, then positions each group meaningfully. This would be especially valuable for larger sessions with many participants, producing more coherent, story-like compositions.
- **Dedicated per-artifact backgrounds for large groups.** When a large session splits into multiple artifacts, the same background is currently reused across all of them. Giving each artifact its own **dedicated, distinct background** would make multi-artifact outputs feel like different scenes. This ties naturally into the AI grouping feature above — related drawings could be paired together and placed on a background dedicated to that group.
- **User-provided drawing labels.** Letting a participant add a one-line caption ("my dog from when I was five") would beat AI-inferred descriptions on accuracy *and* cut Stage 2 cost — exactly the trade-off raised in discovery.
- **Transcript / sentiment integration.** Once Scribl captures session audio (e.g., via the planned Zoom integration), real transcripts could drive richer, more personal compositions.

- **Smarter placement & "integration."** Today cards are arranged logically; the discovery vision of true integration (the family *walking with* the ice cream *under* the palm tree) is a natural next target.
- **Stroke-level data.** Scribl's Metabase stores every stroke as vector data. This opens the door to animation, live "drawing playback," and joint-aware posing — a distinctive, hard-to-copy feature.
- **Scribl Provided Templates.** Have templated scribble artifacts provided by Scribl designers to influence background generation. This could assist with consistent branding/themes.

Robustness & operations

- **Style consistency** across backgrounds (seed/style controls) so a brand or campaign feels cohesive.
- **Multi-round handling** — currently focused on single-round per the agreed POC scope; combining rounds is a known extension.
- **Evaluation harness** — formalize the "what makes a good artifact" criteria into a repeatable scoring pass for regression-testing prompt changes.

9. Recommended Next Phase

The POC ran as a local, script-based pipeline by design — the right scope to prove the concept. The natural next step is to **operationalize it as a live service inside Scribl's product**, along the architecture envisioned in the SOW.

Phase 2 — Productionize & Integrate (recommended):

1. **Deploy as an API.** Wrap the pipeline behind **API Gateway + AWS Lambda** in Scribl's AWS account (per the SOW architecture): client uploads assets to **S3**, invokes the API, and receives the finished artifact. Secure with an API key for the POC-to-pilot stage.
2. **Live integration.** Hook the API into the end-of-session flow so artifacts generate automatically — eliminating manual host effort entirely.
3. **Add the highest-value input: user captions.** Ship the optional one-line drawing label; it's the cheapest, highest-impact quality lever identified.
4. **Pilot & measure.** Run real sessions through the deployed service, using the built-in cost/time reporting to validate per-artifact economics at scale before committing to features.
5. **Then expand selectively** based on pilot results — transcript integration, true compositional "integration," AI grouping with dedicated multi-scene backgrounds, and the stroke-data animation track as a differentiated bet.

Suggested sequence: Deploy → Integrate → add captions → Pilot & measure → expand. This keeps each step funded by demonstrated value and aligned to Scribl's budget sensitivity.

10. Appendix — Running the Pipeline

Full setup and options are in `README.md`. Quick reference:

```
```bash
```

### One session, end-to-end

```
python masking/run_pipeline.py \
```

```
--session "files/Single Round Games/<uuid> - <prompt>"
```

### With drop shadows and 3 refinement passes

```
python masking/run_pipeline.py \
```

```
--session "files/Single Round Games/<uuid> - <prompt>" \
```

```
--shadow --refine-iterations 3
```

### All sessions, consolidated batch report

```
python masking/run_all_sessions.py
```

```
```
```

Outputs land under `masking/output/<session>/`:

- `compose\_output/composite\_canvas.png` — the final artifact
- `pipeline\_report.md` — per-stage timing and cost
- `descriptions.json`, `composition\_plan.json`, `background.png` — intermediate artifacts for inspection

Prerequisites: Python 3.11+, AWS Bedrock access in `us-east-1`, and model access enabled for the configured Claude and Stability AI models.

Prepared by Mission Cloud Services, Inc. — Confidential.

© 2026 Mission Cloud Services, Inc. Proprietary & Confidential.